Tema: Polimorfismo, Clases Abstractas e Interfaces en Java

Esta guía deja la interfaz gráfica para concentrarse en la lógica orientada a objetos el objetivo es crear clases abstractas, aplicar herencia y ejecutar polimorfismo dinámico para gestionar distintos tipos de recompensas.

explica que una clase abstracta funciona como plantilla que no puede instanciarse directamente y que puede contener métodos concretos y abstractos. El polimorfismo permite invocar el mismo método a través de una referencia común y obtener comportamientos distintos según la clase real del objeto. Los errores advertidos son: intentar instanciar la clase abstracta, olvidar @Override al sobrescribir (que sirve como red de seguridad del compilador) y no llamar a super(...) cuando el padre carece de constructor vacío.

Las actividades definen ItemRecompensa como clase abstracta (con nombre, costoPuntos y el método abstracto aplicarEfecto()), junto con las subclases ItemAvatar e ItemPaseAcademico, cada una con su propio atributo y su versión de aplicarEfecto(). Luego, PrincipalPrueba crea un arreglo de tipo ItemRecompensa, almacena objetos de ambas subclases y los recorre con un for invocando el método de forma genérica, demostrando el polimorfismo (mismo mensaje, comportamiento distinto).

Archivo ItemRecompensa.java

```java
public abstract class ItemRecompensa {

    protected String nombre;
    protected int costoPuntos;

    // Constructor
    public ItemRecompensa(String nombre, int costoPuntos) {
        this.nombre = nombre;
        this.costoPuntos = costoPuntos;
    }

    // Método abstracto
    public abstract void aplicarEfecto();

}
```

Es nuestra clase principal abstracta. Guarda el nombre y el costo de las recompensas. También declara el método que las demás clases deberán implementar.

Archivo ItemAvatar.java

```java
public class ItemAvatar extends ItemRecompensa {

    private String tipoAvatar;

    public ItemAvatar(String nombre, int costoPuntos,
                      String tipoAvatar) {

        super(nombre, costoPuntos);
        this.tipoAvatar = tipoAvatar;
    }

    @Override
    public void aplicarEfecto() {

        System.out.println("¡Avatar desbloqueado!");
        System.out.println("Tipo de avatar: " + tipoAvatar);
    }
}
```

Esta clase hereda de ItemRecompensa. Su función es desbloquear un avatar y mostrar qué tipo de avatar se ha obtenido.

ItemPaseAcademico.java

```java

public class ItemPaseAcademico extends ItemRecompensa {

    private String beneficio;

    public ItemPaseAcademico(String nombre, int costoPuntos,
                             String beneficio) {

        super(nombre, costoPuntos);
        this.beneficio = beneficio;
    }

    @Override
    public void aplicarEfecto() {

        System.out.println("¡Pase académico activado!");
        System.out.println("Beneficio: " + beneficio);
    }
}

```

Esta clase también hereda de la clase principal, pero su función es activar un pase académico y mostrar el beneficio correspondiente.

PrincipalPrueba.java

```java
public class PrincipalPrueba {
    Run | Debug
    public static void main(String[] args) {
        // Arreglo de tipo clase abstracta
        ItemRecompensa[] recompensas = new ItemRecompensa[2];
        // Guardamos objetos de diferentes subclases
        recompensas[0] = new ItemAvatar(
            nombre: "Skin de Avatar", costoPuntos: 100, tipoAvatar: "Guerrero"
        );

        recompensas[1] = new ItemPaseAcademico(
            nombre: "Pase de Tarea", costoPuntos: 200, beneficio: "Entrega de tarea adicional"
        );

        // Recorremos el arreglo
        for (int i = 0; i < recompensas.length; i++) {

            System.out.println("---------------------------");

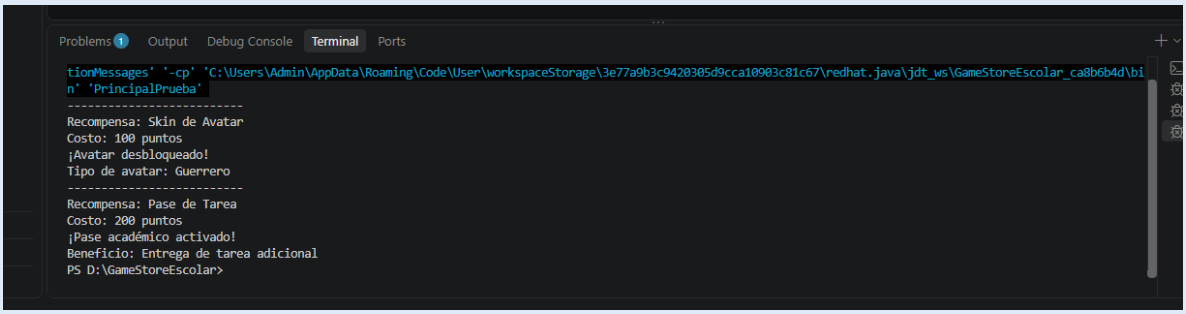
            System.out.println("Recompensa: "
                + recompensas[i].nombre);

            System.out.println("Costo: "
                + recompensas[i].costoPuntos + " puntos");

            // Aplicamos el efecto de cada recompensa
            recompensas[i].aplicarEfecto();
        }
    }
}
```

Crea un arreglo que almacena diferentes tipos de recompensas y utiliza un ciclo for para recorrerlas. Al llamar al mismo método aplicarEfecto(), cada recompensa realiza una acción diferente. Esto demuestra el polimorfismo.

RESULTADO


Problems 1
Output
Debug Console
Terminal
Ports
tionMessages' '-cp' 'C:\Users\Admin\AppData\Roaming\Code\User\workspaceStorage\3e77a9b3c9420305d9cca10903c81c67\redhat.java\jdt_ws\GameStoreEscolar_ca8b6b4d\bi
n' 'PrincipalPrueba'

Recompensa: Skin de Avatar
Costo: 100 puntos
¡Avatar desbloqueado!
Tipo de avatar: Guerrero

Recompensa: Pase de Tarea
Costo: 200 puntos
¡Pase académico activado!
Beneficio: Entrega de tarea adicional
PS D:\GameStoreEscolar>